AGENT TEAM · 架構演進實錄

Code 是 AI 寫的，就真的有一支開發團隊嗎？

從一個 Chat，到一間 AI Company

單一 Chat → 臨時 Sub-agent → 專責 Agent Team → Herdr AI Company

Ian Chen · Microsoft AI MVP · Microsoft Certified Trainer

SPEAKER · 講者介紹



陳葵懋

Ian Chen

一等一科技　研發部主任工程師
Microsoft Certified Trainer (MCT)

15 YEARS
MICROSOFT AI MVP

iangithub.github.io

企業 AI 與軟體架構實務

軟體設計與開發經驗，長期專注 .NET、Azure、容器化與生成式 AI，擅長將企業系統、雲端平台與 AI 技術整合為可上線的解決方案。

20+ 年軟體開發經驗

1,000+ 培訓學員

百萬+ Chatbot 使用者

6+ 本共同著作

專長領域：Azure OpenAI、生成式 AI、RAG、AI Agent、MCP、.NET / C#、Azure / DevOps、Docker / Kubernetes、Agent Framework、企業系統整合

演講與培訓

- 研討會講者：Microsoft TechDays、DevDays Asia、Modern Web、Hello World、MOPCON、iThome AI & Cloud
- 企業、公務機關與大學院校 AI 技術培訓（司法院、國教署）

代表成果

- 疫情期間開發「敗口罩」與「找找快篩試劑」LINE Chatbot，兩項專案累計服務百萬以上使用者
- 共同著作 6+ 本：ChatGPT、LangChain、AI Agent、MCP

OPENING · 我自己的數字

5.3 萬 行程式碼

2 週內，AI 幫我寫的

AI 寫 code，已經夠快了。

下一題：誰來審、誰來測、誰說了算？

當 Agent 從一個變成一群，協作架構決定交付能不能被信任。今天用四個階段，走一遍我自己的 Agent Team 演進。

ROADMAP

四次演進，一條路線

01

單一 Chat

一個人包辦全部

02

臨時 Sub-agent

一句 prompt 叫助手幫忙

03

專責 Agent Team

主控照 SOP 派給專責同事

04

Herdr AI Company

獨立員工，跨 CLI 依公司制度協作

今天的重點

LEVEL 01

單一 Chat：一個對話包辦全部

- 需求、程式碼、測試 log、審查意見，全部堆在同一條對話
- 同一個模型、同一套權限、同一份上下文
- 對話越拉越長，一開始講好的決定慢慢被稀釋

適合：釐清需求、快速原型、範圍明確的小修改



LEVEL 01 · 撞牆

單一 Chat 撞到的三面牆

上下文污染

測試 log、堆疊追蹤把需求與決策淹沒，對話越長越不可靠。

自己審自己

寫程式的視角就是審查的視角，盲點會一路帶過去。

權限只有一套

會改檔的 Agent 同時在做審查，無法強制「只讀」。

這三面牆，就是接下來每一次演進要拆掉的東西。

LEVEL 02

臨時 Sub-agent：一句 prompt 叫出助手

子 Agent 沒有事先設計：扮演什麼角色、怎麼做、交回什麼，全看當下這句 prompt

在 main chat 裡隨口一句：

「請用 subagent 幫我做 code review」





- 藍點：委派任務（往下）
- 橘點：回傳摘要（往上）

用的是 CLI 內建的通用型 Agent

Codex：default · worker · explorer

Claude Code：general-purpose 等

LEVEL 02 · 盤點

臨時 Sub-agent 解決了什麼，還留下什麼

解決了

- ✓ 雜訊留在子 Agent，主對話保持乾淨
- ✓ 一句 prompt 就能分工、平行處理
- ✓ Codex、Claude Code 內建，不用任何設定

還留下

- ⚠ 子 Agent 沒有專屬角色，審查標準看當下 prompt 怎麼寫
- ⚠ 沒另外指定，就沿用主 Agent 的模型與推理強度
- ⚠ 派給誰、做完怎麼驗收，每次由主 Agent 臨場決定

LEVEL 03

主控 Agent 依 Skill，派工給專責 Agent

每位 Agent 事先設計好角色指令、模型與 Skill；主 Agent 照 Team Skill 的 SOP 依序派工

Main Agent

Team Skill · SOP

1. **需求討論、產生 spec**
   Main · OpenSpec
2. **實作，build / test 必過**
   → coding-agent
3. **程式審查**
   → code-review-agent
4. **Fortify 預檢**
   → fortify-preverify
5. **彙整回報使用者**
   Main

Coding

coding-agent

指令	依 spec 實作
Skill	coding SOP
交件	build／test 通過
模型	依工作指定

Code Review

code-review-agent

指令	依 checklist 審 diff
Skill	review SOP
交件	審查報告
順序	Coding 之後

Fortify 預檢

fortify-preverify

指令	對照 Fortify 分類
Skill	fortify rules
交件	預檢報告
順序	Review 之後

主 Agent 照 SOP 找對的人，不用再臨場決定誰做什麼

我的實作：主 Agent 用工具內建 Agent，3 個自訂子 Agent＋4 個共用 Skills，Review 與 Fortify 串列審查

LEVEL 03 · 盤點

設計好了，但還關在同一個 CLI 裡

同一個 Codex CLI 主 Session

Main Agent

`coding` `code-review` `fortify`

共用同一套 Sandbox 與核准設定

Claude Code

無法加入這個團隊

01 只能在同一個 CLI 裡組隊，做不到「Claude Code 寫、Codex 審」

02 Codex 自訂 Agent 繼承主 Session 的 Sandbox 與核准設定

03 為了讓 Coding 能 restore 而放寬網路，Review 也沿用同一套設定

04 執行過程收在各自的 Agent Thread，要逐一點開查看

補充：Claude Code 另有 Agent Teams（Experimental），成員可直接互傳訊息、共用任務清單，但仍只限 Claude Code Session。

來源：OpenAI Codex Subagents、Claude Code Agent Teams 官方文件（2026-10-30 查閱）

LEVEL 04

Herdr AI Company

跨 CLI 的團隊與部門管理

把 Level 3 設計好的專責 Agent，各自搬進獨立的 CLI Session 與 Pane。主管依 Team Skill 派工，依 Contract 與 Quality Gate 驗收。

WHAT IS HERDR

什麼是 Herdr？

“the runtime your coding agents live on.” *官方標語*

開源的 Agent 多工器：一個 Rust 執行檔，在你原本的終端機裡，替每個 Coding Agent 開一個真的 Pane。

</> 真實終端機 Pane

每個 Agent 跑在自己的 Pane；Herdr 管終端機，不包裝 Agent

看得懂 Agent 狀態

working · blocked · idle · done，一眼看出誰在等你

常駐伺服器

關掉視窗或 SSH 斷線，Agent 照跑；再執行 herdr 就接回來

Agent 能指揮 Agent

CLI 與 Socket API：開 Pane、送 Prompt、等待狀態、讀輸出

跨 CLI、跨機器

Claude Code、Codex、Cursor、OpenCode，加上 SSH 主機，同一個視窗

可擴充

官方整合、Agent Skill、Plugin

Apache 2.0 開源 · macOS／Linux／Windows（官方標示 Beta）· 最新版 v0.9.5（2026-10-14）

來源：herdr.dev、github.com/herdrdev/herdr（2026-10-31 查閱）

01 單一 Chat　02 臨時 Sub-agent　03 專責 Agent Team　**04 Herdr AI Company**

LEVEL 04 · 實際畫面

一個畫面，看見整個團隊



真實終端機 Pane

每位員工一個獨立 CLI 程序

狀態偵測

idle · working · blocked · done

Agent 也能操控 Agent

agent start · prompt · wait · read

跨 CLI

Codex、Claude Code 可並存

LEVEL 04 · 分工

Herdr 搭好舞台，制度要自己蓋

Quality Gate	依同一版本的有效測試與審查證據，判定是否完成
Communication Contract	派工、回報、修復的固定資料格式
Worker Skill + Project Rules	每位員工的 SOP，加上專案自己的規範
Team Skill	團隊編制、執行順序、重試流程
Herdr Runtime	Pane、Agent 啟動、狀態偵測、Prompt 與輸出讀取

AI Company 自建

上面四層，都是我們自己寫的規則與流程

Herdr 提供

01 單一 Chat　02 臨時 Sub-agent　03 專責 Agent Team　04 Herdr AI Company

LEVEL 04 · 組織

AI Company 的組織圖



COMPANY BLUEPRINT

公司藍圖：三個區域，各管一件事

01 · 在這裡開工

D:\Workspaces\

專案現場

AGENTS.md	宣告部門
docs\team-rules\	專案規範
src\ · tests\	實作時建立
drafts\ · examples\	書稿與範例

與公司級分離

02 · 啟動時經過

C:\Users\<你>\

啟動橋接

.codex\AGENTS.md	Codex 短指引
.claude\CLAUDE.md	Claude 短指引
.agents\skills*	Junction
.claude\skills*	Junction
herdr	官方安裝維護

越薄越好，只做橋接

03 · 制度住在這裡

D:\AI-Company\

公司制度

.agents\skills\	12 個 Skill
bootstrap\	Manager 啟動規則
contracts\	契約・Schema
policies\	權限與交付政策
scripts\	工作流水線

唯一來源，只裝一份

公司制度只裝一份，專案只宣告部門，使用者層級只做橋接。

01 單一 Chat　02 臨時 Sub-agent　03 專責 Agent Team　04 Herdr AI Company

D:\AI-Company\.agents\skills\

12 個 Skill，分成三層

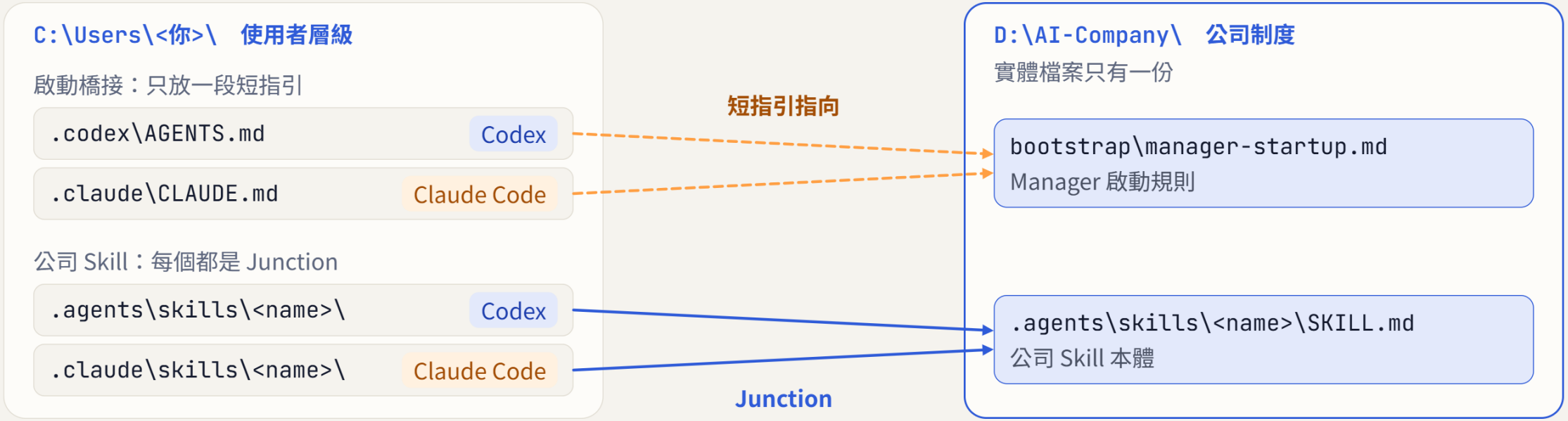


Team Skill 決定誰、何時做；Worker Skill 決定怎麼做。

命名一律「部門-角色」：灰色是部門，白色是角色，一眼看出歸屬

SINGLE SOURCE OF TRUTH

一份 Skill，Codex 與 Claude Code 共用



實體只有一份：改 D:\AI-Company 一次，所有專案與兩種 CLI 同步生效。

下一張：實機畫面與建立指令

JUNCTION · 實機畫面

同一個資料夾，三種來源，各自維護

C:\Users\ianch\.agents\skills　　使用者全域 Skill

herdr	實體資料夾	官方安裝・官方維護
software-code-review	Junction	公司 Skill
tech-book-Reviewer	實體資料夾	個人 Skill・自己維護

D:\AI-Company\.agents\skills\
software-code-review
實體只有一份，改這裡就同步

PowerShell 7　　Junction 不需要系統管理員權限

```
❯ $Src = "D:\AI-Company\.agents\skills\software-code-review"
❯ $Dst = "$HOME\.agents\skills\software-code-review"
❯ New-Item -ItemType Junction -Path $Dst -Target $Src
  l----  software-code-review → D:\AI-Company\.agents\skills\software-code-review
❯ Get-Item -LiteralPath $Dst | Format-List FullName,LinkType,Target
  LinkType : Junction
  Target   : D:\AI-Company\.agents\skills\software-code-review
```

BOOT SEQUENCE

開一個 CLI，怎麼變成 Manager？

1. **在專案目錄開 CLI**
   在 D:\Workspaces\… 啟動 Codex 或 Claude Code
2. **讀到使用者層級的短指引**
   AGENTS.md／CLAUDE.md 指向 manager-startup.md
3. **依啟動規則，成為 Manager**
   manager-startup.md 規定 Manager 怎麼開工
4. **讀專案 AGENTS.md**
   Department 定主責部門；Rule Registry 定要載入的規則
5. **啟動 Team Skill，開始派工**
   Pane 操作依 runtime.md；確認 Gate 腳本後派工



LEVEL 04 · 能力

同一個 Skill，依專案載入不同 Rules

Company Root 共用

Worker Skill

員工的 SOP：步驟、輸出格式、回報方式

+

Project Root 各自

Project Rules

docs\team-rules，由專案 AGENTS.md 的 Rule Registry 指定

=

可交付

這個專案的員工

照公司流程做事，照專案規矩交件

實例 · 同一套公司 Skill，套到不同專案

Project-A	Code Review Skill	+	architecture · coding · testing	→ Project-A 的 Reviewer
Book-A	Fact Checker Skill	+	editorial · fact-check · code-examples	→ Book-A 的事實查核員

LEVEL 04 · 編制

每位員工：自己的模型、權限與職責

Coding Agent

`software-coding`

`WORKSPACE-WRITE`

`gpt-6-sol · high`

負責正式程式碼修改
可執行 build、restore、test
網路：開啟
不得寫出工作區

Unit Test Agent

`software-unit-test`

`WORKSPACE-WRITE`

`gpt-6-luna · high`

獨立驗證 build 與測試
只驗證，不改正式程式碼
網路：開啟
失敗回報給 Manager

Code Review

`software-code-review`

`READ-ONLY`

`gpt-6-astra · high`

只審不修，修正交回 Coding
發現分 blocking／non-blocking
網路：關閉
高風險變更改用 xhigh

Repo Scout

`software-repo-scout`

`READ-ONLY`

`gpt-6-luna · medium`

相依追蹤與影響分析
只在必要時啟動
網路：關閉
不實作任何變更

唯讀角色由 Sandbox 強制執行；Git commit 只由 Manager 在使用者明確要求時處理。

設定取自 herdr-team Skill：approval policy 一律 never，禁用 danger-full-access 與 --yolo

CONTRACT-DRIVEN ORCHESTRATION

交接有格式，驗收看證據

1

Task Dispatch

任務派發

帶著需求、驗收標準、限制與 repo 基準狀態出發

2

Worker Report

工作回報

PASS FAIL BLOCKED UNVERIFIED

附執行指令、結果與證據

3

Repair Request

修復要求

保留 Finding ID、證據與修正要求，只針對未通過項目

4

Quality Gate

品質關卡

必須取得同一版本的有效報告，才能通過驗收

D:\AI-Company

制度寫成檔案與程式

contracts\

JSON 契約 · Schema · 狀態機

scripts\

Snapshot → Contract → Repair → Gate

01 單一 Chat　02 臨時 Sub-agent　03 專責 Agent Team　04 Herdr AI Company

QUALITY LOOP

Coding → Test → Review，直到沒有 blocking



Reviewer 只審不修

發現的問題一律交回 Coding Agent 修正，再由 Test Agent 重新驗證

Manager 負責裁決

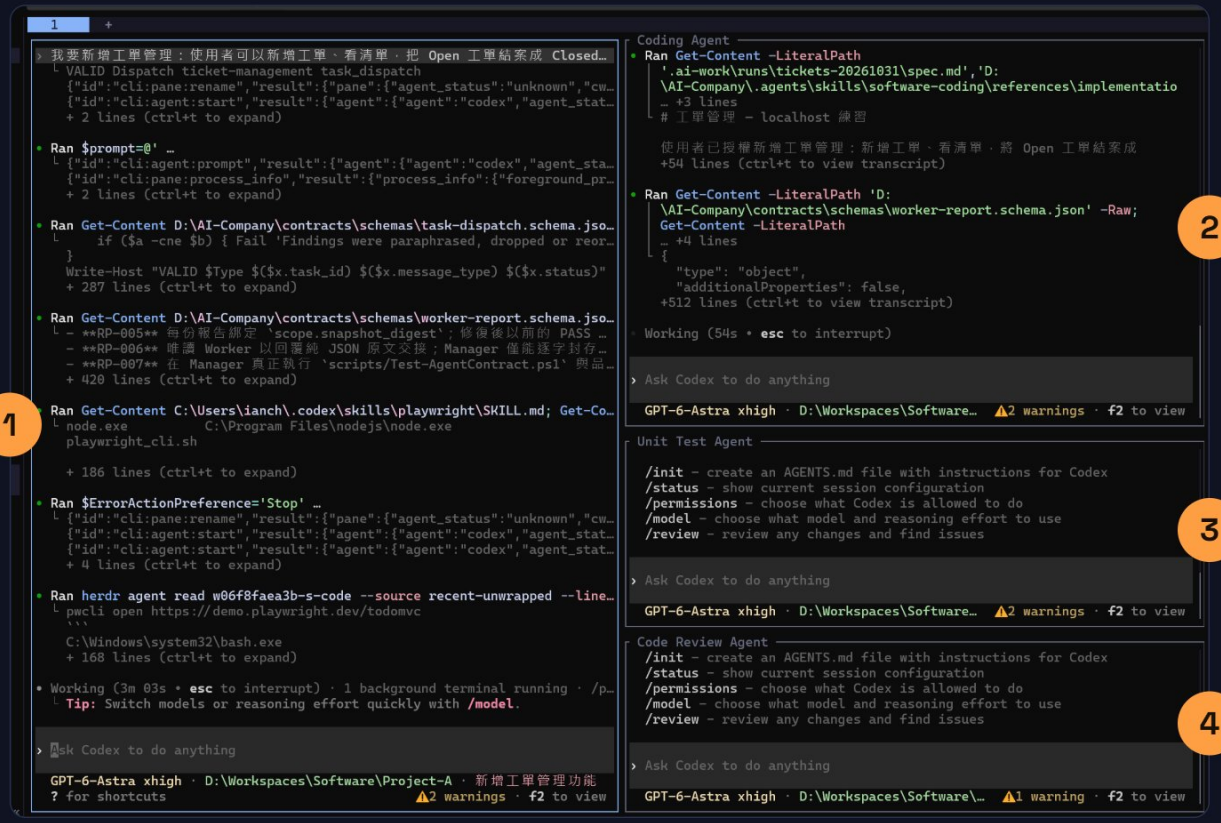
同一個 blocking 反覆出現或意見衝突，依需求與 repo 證據裁決，避免無限迴圈

non-blocking 不拖住交付

只在明確改善、不擴大範圍時採用，其餘回報給使用者

LIVE · 實機畫面

實戰：一句需求，四個 Pane 同時開工



1 **Manager Pane**
收到「新增工單管理」，Task Dispatch 先通過契約驗證，再用 herdr CLI 改名、啟動、派工

2 **Coding Agent**
讀 run 目錄的 spec.md、software-coding Skill 與 worker-report schema，開始實作

3 **Unit Test Agent**
待命：等 Coding 交件，再獨立驗證

4 **Code Review Agent**
待命：只審不修，問題退回 Coding

誰在做事、讀了哪份規則，全部攤在畫面上

LIVE · 契約驗證

契約會擋人：Findings 被改寫就驗證失敗

契約驗證腳本（實機畫面節錄）

```
if ($a -cne $b) { Fail 'Findings were paraphrased, dropped or reor…'  ← -cne 逐字比對
Write-Host "VALID $Type $($x.task_id) $($x.message_type) $($x.status)"  ← 通過才印出 VALID
```

實際輸出

```
VALID Dispatch ticket-management task_dispatch
```

契約規則（節錄）

- **RP-005** 報告綁定 snapshot_digest，修復後要重新驗證
- **RP-006** 唯讀 Worker 交純 JSON 原文，Manager 只能逐字封存
- **RP-007** Manager 必須真正執行驗證腳本

worker-report.schema.json

```
"additionalProperties": false
```

多一個欄位，報告就不合法

交接內容由程式把關，Manager 想「幫忙整理」也過不了關。

TRADE-OFF

多 Agent 團隊：換來什麼，付出什麼

換來 GAIN

- 專業分工
- 獨立驗證
- 更高的流程可控性

vs

代價 COST

- 更多 Token
- 協調成本
- 執行時間
- 環境複雜度

每往上一層，都要問：**換來的，值不值得這些代價？**

COMPARISON

四種做法，一張表看差異

評估項目	Single Agent	Agent + Skill	Native Sub-agent	Herdr AI Team
入門難度	最低	低	中	高
初始 Token 開銷	較低	低～中	中～高	通常較高
SOP 標準化	需自行規範	強	強，可搭配 Skill	強，可搭配 Team / Worker Skill
Agent 專業分工	單一角色	可模擬多角色	原生支援	獨立 CLI 分工
上下文隔離	無	主要仍是同一 Context	獨立 Context	各 CLI 獨立 Context
跨模型協作	單 Session 選擇模型	同左	同一 CLI 支援的模型	可跨 Codex、Claude 等
執行可視化	單一對話	單一對話	Agent Thread / 任務介面	每個 Agent 獨立 Pane
Agent 生命週期	簡單	簡單	CLI 管理	Manager / Herdr 管理
Agent 間協作	不需要交接	角色切換	原生委派、回傳	可自訂標準交接契約
建置與維護成本	最低	低～中	中	最高
適合任務	小型、明確需求	可重複的標準流程	可拆解的複雜任務	長期、跨工具、需獨立驗證的任務

HEAD TO HEAD

Herdr 與原生 Sub-agent 的差異

Herdr + 獨立 CLI Agent	VS	Codex / Claude 原生 Sub-agent
外部 Multi-Agent Orchestration	架構定位	CLI 內部 Agent Delegation
每個 Pane 啟動獨立 CLI Session	Agent 執行方式	主 Agent 啟動 Sub-agent
各 CLI 獨立 Context	上下文	各 Sub-agent 也有獨立 Context
可以混用 Codex、Claude Code	跨 CLI 協作	主要在所屬 CLI 生態內
各 Pane 獨立指定模型	模型選擇	可依 CLI 支援能力指定模型
每個 CLI 可以獨立配置 Sandbox	權限	依各 CLI 的 Sub-agent 權限機制
每個 Worker 執行自己的 Skill	Skill	Sub-agent 也能使用 Skill
透過 Herdr 與自訂 Contract	Agent 溝通	使用 CLI 原生委派與回報機制
各 Agent 有獨立 Pane	可觀察性	由 CLI 的 Agent 介面觀察
通常較高，可能重複載入上下文	Token 成本	也有額外成本，但原生整合較簡單
高，需管理團隊與交接	建置複雜度	相對較低
長期、跨工具、專業分工的團隊	適合情境	單一 CLI 的任務拆解與平行工作

HONEST TAKE

Herdr 沒有全面勝出

原生 Sub-agent 已經做得到

- ✓ 上下文隔離
- ✓ 不同模型配置
- ✓ 工具權限控制
- ✓ 專業分工

而且通常不需要另外管理多個 CLI 程序

Herdr 的優勢集中在這四項

- ☆ 跨 CLI
- ☆ 獨立 Session
- ☆ 可觀察性
- ☆ 工作團隊管理

真的需要右邊這四項，才值得多管一層 CLI。

DECISION GUIDE

工程師到底該怎麼選？

實際工作情境	比較建議的模式	原因
修改一個小 Bug	Single Agent	成本最低，協調最少
標準化 Coding 流程	Agent + Skill	SOP 一致，不增加多代理開銷
大量探索程式庫	Native Sub-agent	可平行探索、回傳摘要
多面向獨立 Code Review	Native Sub-agent 或 Agent Teams	不一定需要跨 CLI
要 Claude 寫、Codex 審查	Herdr AI Team	跨工具模型協作
長期維護的大型專案	Herdr AI Team	可持續管理不同專業角色
需要明確驗收及修復流程	Skill + Contract；可搭配 Herdr	重點在品質制度，不只在 Agent 數量
經常需要人介入不同 Agent	Herdr AI Team	每個 Agent 的工作過程容易直接觀察

BEFORE YOU SHIP

落地前，先知道這四件事

01 Pane 不等於檔案隔離

多位 Worker 共用同一個工作目錄，仍然會搶同一個檔案。用檔案所有權、Worktree 或 Sandbox 切開。

02 Herdr 不幫你驗收

Herdr 提供 Pane、啟動、狀態與讀取；Contract 與 Quality Gate 要自己設計、自己維護。

03 建置成本比較高

Team Skill、Worker Skill、Contract 都是要維護的資產。先從小專案打模，再慢慢修順。

04 Worker 先不開 Sub-agent

第一階段避免兩層看不見的調度。公司層級流程穩定後，再依需要啟用。

TAKEAWAY

原生 Sub-agent

一位工程師，找幾位助手完成工作。

Herdr AI Company

一位主管管理多位獨立員工，按照公司制度完成交付。

先從一個小專案打模，再一步一步修順。

Ian Chen ・ Microsoft AI MVP ・ 謝謝聆聽，歡迎提問

本次簡報

GitHub ・ mopcon-2026

追蹤我的 Facebook

facebook.com/ian.no18